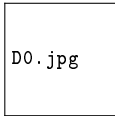


Lecture 34: Advanced SQL Finale

Analytical Moving Windows, Stored Procedures, and Triggers

Z2004: Database Management Systems



D0.jpg

IIT Madras Zanzibar

Friday, 12 June 2026

The Absolute Pinnacle of Database Programming

- ➊ 5 Question Pop Quiz on Window Ranking.
- ➋ Partitioned Windows: `PARTITION BY` and inline metrics calculations.
- ➌ Time Travel Functions: Tracking shifts via `LAG()` and `LEAD()`.
- ➍ Stored Procedures: Packaging logic using `CREATE PROCEDURE`.
- ➎ Database Triggers: Active automation via `AFTER` and `INSTEAD OF`.

Window Functions I Quiz

Write the exact SQL queries for the SchoolDB.

Tables: *Student*(SID, Name, Dept, GPA).

- ❶ **Sequencing:** Write a query that utilizes a window function to assign a strict sequential row ordering index to every student tuple sorted by their GPA descending.
- ❷ **Ranking gaps:** If three records tie for position 2 using the `RANK()` function, what numerical position sequence identifier is assigned to the next subsequent record?
- ❸ **Tight Ranking:** Which specific analytical ranking window function guarantees that sequence positions increment incrementally by 1 without leaving gaps after ties?
- ❹ **Grouping Layout:** Write a query utilizing the `NTILE` function to break the entire student table body into exactly three performance tracking layers based on GPA.
- ❺ **The Operator:** What mandatory syntactic keyword must follow a windowing function call to attach it to an internal processing space specification?

🕒 You have 7 Minutes. No notes, pure absolute silence!

1. Strict Sequencing

```
SELECT Name ,  
       ROW_NUMBER() OVER (ORDER BY GPA DESC)  
FROM Student ;
```

2. Ranking Sequence Gaps

- **Answer:** Position **5**. Three rows share position 2 (occupying slots 2, 3, and 4), so the system skips directly to 5.

3. Tight Ranking Sequence

- **Answer:** DENSE_RANK()

4. Grouping Layout

```
SELECT Name , GPA ,  
       NTILE(3) OVER (ORDER BY GPA DESC)  
FROM Student ;
```

5. The Windowing Operator

- **Answer:** The OVER keyword.

Window Functions II: PARTITION BY

OVER(...) defines the window; PARTITION BY splits rows into independent groups and runs the same window calculation *separately* per group.

```
SELECT Name, Dept, GPA,
       AVG(GPA) OVER (PARTITION BY Dept) AS DeptAverage,
       SUM(Credits) OVER (PARTITION BY Dept) AS DeptTotalCredits
FROM Student s JOIN Enrolment e ON s.SID = e.SID JOIN Course c ON e.CID = c.CID;
```

Partitioned output (each Dept is its own mini-table):

Name	Dept	GPA	DeptAverage	What changed?
Asha Ali	CSE	3.80	3.86	Avg only within CSE partition
Noah Kim	CSE	3.92	3.86	Same CSE partition, same average
Zain Omar	EEE	3.45	3.45	New partition (EEE), average recalculates

OVER() vs OVER(PARTITION BY ...) (the key difference)

No partition (whole result is one window)

```
SELECT Name, Dept, GPA,  
       AVG(GPA) OVER () AS SchoolAvg  
FROM Student;
```

Name	Dept	GPA	SchoolAvg
Asha	CSE	3.80	3.72
Noah	CSE	3.92	3.72
Zain	EEE	3.45	3.72

Partitioned (one window per Dept)

```
SELECT Name, Dept, GPA,  
       AVG(GPA) OVER (  
         PARTITION BY Dept  
       ) AS DeptAvg  
FROM Student;
```

Name	Dept	GPA	DeptAvg
Asha	CSE	3.80	3.86
Noah	CSE	3.92	3.86
Zain	EEE	3.45	3.45

Takeaway: PARTITION BY changes the *scope* of the window calculation (global vs per-group).

Dept average GPA: 3 ways (why window functions are better)

- Goal: keep **each student row** and also show the **department average**.

(A) GROUP BY + JOIN

```
SELECT s.Name, s.Dept,
       s.GPA, d.DeptAvg
FROM Student s
JOIN (
  SELECT Dept,
         AVG(GPA) AS DeptAvg
  FROM Student
  GROUP BY Dept
) d ON d.Dept = s.Dept;
```

(B) Correlated subquery

```
SELECT s.Name, s.Dept,
       s.GPA,
       (SELECT AVG(s2.GPA)
        FROM Student s2
        WHERE s2.Dept=s.Dept
       ) AS DeptAvg
FROM Student s;
```

(C) Window function

```
SELECT Name, Dept,
       GPA,
       AVG(GPA) OVER (
         PARTITION BY Dept
       ) AS DeptAvg
FROM Student;
```

Why windows are better: you keep the same rows as Student, but can still compute group metrics *without* an extra join/subquery layer.

Time Travel: LAG() and LEAD()

Accessing values from neighboring rows without self-joins

LAG(): looks backward at previous rows. LEAD(): looks forward at upcoming rows.

Why we use them: compute *differences to the previous/next row* (e.g., "how far behind the next higher GPA am I?") without a self-join.

```
-- Order students by GPA and compute the gap to the next-higher student
SELECT Name, GPA,
       LAG(Name) OVER (ORDER BY GPA DESC) AS PersonAhead,
       LEAD(Name) OVER (ORDER BY GPA DESC) AS PersonBehind,
       LAG(GPA) OVER (ORDER BY GPA DESC) - GPA AS GapToAhead
FROM Student;
```

Output Frame Matrix:

Name	GPA	PersonAhead	PersonBehind	GapToAhead
Noah Kim	3.92	NULL	Asha Ali	NULL
Asha Ali	3.80	Noah Kim	Lina Zhang	0.12
Lina Zhang	3.74	Asha Ali	Arjun Singh	0.06

image.png

Stored Procedures (what/why)

A stored procedure is a named program saved in the database

- ▶ **What:** SQL statements (and control flow) stored on the **DB server** and executed via a name.
- ▶ **Inputs:** can take **parameters** (e.g., dept code, min GPA).
- ▶ **Why:**
 - **Reuse & consistency:** one definition used by many apps/users
 - **Security:** grant EXECUTE without exposing underlying tables
 - **Less app-side SQL:** call EXEC ... instead of sending big queries

Call pattern:

```
EXEC sp_GetDepartmentRoster @DepartmentCode='CSE', @MinGpa=3.5;
```

Stored Procedures: Encapsulating Execution

Functions stored directly in the database engine server Catalog

Instead of allowing frontend web apps to write raw queries, we package SQL code blocks into **Stored Procedures** that accept parameterized input values.

```
-- Create a modular parameter-driven execution block
CREATE PROCEDURE sp_GetDepartmentRoster
    @DepartmentCode VARCHAR(10),
    @MinGpa          DECIMAL(3,2)
AS
BEGIN
    SELECT Name, GPA, Email
    FROM Student
    WHERE Dept = @DepartmentCode AND GPA >= @MinGpa
    ORDER BY GPA DESC;
END;
```

Invoking the Code Block:

```
EXEC sp_GetDepartmentRoster @DepartmentCode = 'CSE', @MinGpa = 3.5;
```

Stored Procedures: What happens when we EXEC?

Example call:

```
EXEC sp_GetDepartmentRoster @DepartmentCode = 'CSE', @MinGpa = 3.5;
```

Effect: runs the stored SELECT *on the server*, returning a filtered + sorted result set.

Example output:

Name	GPA	Email
Noah Kim	3.92	noah.kim@univ.edu
Asha Ali	3.80	asha.ali@univ.edu

Automatic actions attached to table events

- ▶ **Created manually:** a DBA/developer writes `CREATE TRIGGER ...` once; after that it runs **automatically**.
- ▶ **When it fires:** on `INSERT/UPDATE/DELETE` (and in some DBMSs, also DDL events).
- ▶ **Kinds:** `BEFORE / AFTER / INSTEAD OF` (support depends on DBMS).
- ▶ **How many?** typically **multiple triggers per table/event** are allowed; firing order can be DBMS-specific.

Mental model: your DML statement → DB applies the change → trigger code runs (or intercepts it).

Database Triggers: Event Automation

Event-driven scripts that execute automatically on data modification

A **Trigger** fires automatically when a DML action (INSERT, UPDATE, DELETE) hits a table. It uses virtual validation tables named `inserted` and `deleted`.

An AFTER Trigger Example:

```
CREATE TRIGGER trg_LogStudentInsert
ON Student
AFTER INSERT
AS
BEGIN
    PRINT 'Security Event: A new student record has
        been added.'
    SELECT Name, Email FROM inserted;
END;
```

Fires **after** data checks pass. Excellent for building audit trails.

An INSTEAD OF Trigger Example:

```
CREATE TRIGGER trg_PreventDeletes
ON Student
INSTEAD OF DELETE
AS
BEGIN
    RAISERROR('System Policy: Hard deletions are
        strictly blocked.', 16, 1);
END;
```

Intercepts the query entirely. Used to implement soft-deletes or override actions on views.

Why triggers matter (what they help with)

- ▶ **Audit trails:** log who/what/when changed a row (security + compliance).
- ▶ **Enforcing business rules:** block invalid operations or auto-fix data.
- ▶ **Derived data / counters:** maintain summary tables (use with care).
- ▶ **Soft deletes:** convert DELETE into UPDATE IsDeleted=1.

Cautions (why not overuse):

- ▶ Hidden side effects: logic runs **outside** application code.
- ▶ Debugging/performance: can add extra work on writes; test carefully.
- ▶ Keep them **small, documented, and predictable**.

Why data + DBMS + SQL matter

- ▶ **Data is the product:** decisions, personalization, automation, and AI all run on reliable data.
- ▶ **DBMS is the infrastructure:** correctness (ACID), concurrency, recovery, security, and performance at scale.
- ▶ **SQL is the interface:** the common language to query, clean, aggregate, and enforce constraints.
- ▶ **Where you'll use this:** fintech ledgers, e-commerce orders, hospital records, IoT telemetry, analytics dashboards, recommendation systems.
- ▶ **Your edge:** designing schemas + writing efficient queries + understanding execution plans = fewer bugs and faster systems.



datra.png

Congratulations! You have completed Database Management Systems!

What You Have Mastered:

- ▶ Relational Algebra & Calculus Math Foundation
- ▶ ERD Schema Normalization (1NF $\rightarrow$ BCNF)
- ▶ Relational Storage Internals (B+ Trees, Indexes)
- ▶ Complete Core SQL DDL/DML Fluency
- ▶ Advanced Window, CTE, Procedural Engines

Your Next Engineering Horizon:

- ▶ Application Layer Object-Relational Mapping (ORMs)
- ▶ Enterprise Database Tuning & Query Optimization
- ▶ NoSQL Distributed Clusters (MongoDB, Redis)
- ▶ Big Data Warehouse Systems (Hive, Spark)

Good luck with your upcoming final projects and terminal exams!

WhatsApp Image 2026-06-12 at 09.46.37.jpeg